

Research note

What the learner problem actually is, what already exists, where the gaps are, and the product direction that follows.

1. The learner problem

LLD practice has an unusual shape: it is **easy to start and almost impossible to self-assess**.

A learner can read "design a parking lot", spend forty minutes producing a class diagram, and come away with no idea whether it was any good. Three properties of the domain cause this:

There is no single correct answer. Unlike a DSA problem, where a test suite settles it, a parking lot can put pricing in a `PricingStrategy`, a `RateCard`, or on `Ticket` itself and be defensible in all three. So the obvious feedback mechanism, compare against a reference solution, is actively wrong. It teaches conformity to one author's taste and punishes good designs that differ.

The mistakes are invisible from the inside. The errors that matter are structural: a class doing two jobs, cardinality that cannot express a real case, a switch statement that will be reopened for every new requirement. A learner who makes one of these does not experience an error. Their design looks finished to them. This is precisely the gap a reviewer closes and self-study cannot.

The graded artefact is partly an argument, not an artefact. In a real interview, roughly half the signal is the reasoning: *why this shape and not the other one, what does your choice make harder*. A learner practising alone produces the diagram and skips the argument entirely, because nothing forces it out of them.

The consequence is a plateau. Learners solve Parking Lot, Elevator and Vending Machine, feel no more confident than before, and conclude LLD is a thing you either have a knack for or do not.

2. What already exists

Approach	What it does well	Where it leaves the learner
Blog posts & GitHub repos (low-level-design-primer , awesome-lld lists)	Free, plentiful, often genuinely good reference solutions.	Purely read-only. You compare your answer to theirs and see a difference, with no way to know whether the difference matters. No practice loop at all.
Books & courses (Head First Design Patterns, Grokking the OOD Interview)	Excellent at teaching the vocabulary: patterns, SOLID, when to use which.	Worked examples, not exercises. The reader is never asked to produce a design that is then examined. Knowledge without a feedback signal.
DSA platforms (LeetCode, HackerRank)	The loop is perfect: attempt → automatic verdict → retry, with history.	The verdict comes from test execution, which LLD has no equivalent of. Their LLD-adjacent content is multiple-choice or "here's the answer, compare yourself".
Mock interview marketplaces (Pramp, interviewing.io)	The highest-quality feedback available: a human who probes the reasoning.	Expensive, scheduled, and roughly one problem per session. Practising a design five times to see it improve is not economically possible.
General-purpose LLMs (paste your design into ChatGPT)	Free, instant, surprisingly perceptive on structure.	Three failure modes seen consistently in testing: it is agreeable, praising weak designs; it drifts, giving different verdicts for the same input; and it grades against whatever reference solution it happens to recall, marking down unconventional-but-sound answers. It also never says "you did not mention pricing at all" reliably, because it has no fixed checklist.

3. The gaps worth building into

Gap 1: Nothing closes the loop. Reading material is read-only; mock interviews do not repeat cheaply. Nobody offers *attempt → feedback → attempt again → see the difference* for LLD, which is exactly the loop that made DSA practice work.

Gap 2: Feedback is either rigorous or scalable, never both. A human reviewer is rigorous and does not scale. A raw LLM scales and is not rigorous; it will not reliably notice a missing requirement, and it varies run to run.

Gap 3: Learners are not made to state the parts that get graded. A blank text box lets someone write three fluent paragraphs while never naming a class's responsibility or a relationship's cardinality. The submission format itself is a teaching tool that everyone leaves on the table.

Gap 4: Nothing distinguishes a fact from an opinion. When an LLM says "your Vehicle class violates SRP", the learner cannot tell whether that is checkable or a guess, so they either accept everything or trust nothing.

4. Product direction

DesignDojo: a repeatable practice loop for LLD, with feedback that shows its work.

Four positions follow directly from the gaps above.

1. The submission is structured, not prose. The learner names their classes, writes one line of responsibility for each, and states relationships *with cardinality*. This is the product's main teaching device: it makes the skipped decisions unskippable, and it happens to give the evaluator something precise to read. Code and Mermaid class diagrams are accepted too, for learners who think in those, all three normalise to the same internal model. (*Gap 3.*)

2. Evaluation is deliberately split in two. Checkable things (is each stated requirement addressed, is any class carrying twelve members, is cardinality stated, does the design mention what happens when the lot is full) are decided by reproducible rules. Judgement calls (is this abstraction well chosen, does the reasoning hold) go to a language model, which is given the rule findings and told not to duplicate them. The rubric decides per dimension how much each source is trusted. (*Gap 2.*)

3. Every finding is labelled with its source, on screen. A learner can see that "nothing owns pricing" came from a rule that will show its matched evidence, while "Vehicle is doing two jobs" is a reviewer's opinion they are free to argue with. Requirement coverage displays the learner's own words that matched, so a wrong verdict is arguable rather than authoritative. (*Gap 4.*)

4. Attempts are never overwritten. Every attempt is kept, numbered, and charted. The product's core screen is not the score; it is the second attempt scoring higher than the first on the dimension the learner was told to fix. (*Gap 1.*)

What is deliberately not in the MVP

Accounts and auth (a per-browser learner id is enough to make history real), a problem-authoring UI (problems are seeded data: a mentor could add one without an engineer, which is the point), social or leaderboard features (they reward score-chasing over the retry loop), and rendered diagram editing (Mermaid text diffs between attempts; a canvas does not).

How this would be validated

The metric that matters is **score improvement between attempt 1 and attempt 2 on the same problem**: the loop either works or it does not. Secondary: what share of learners submit a second attempt at all, and whether the dimension a learner was told to fix is the one that moves.